To recognize

if statements and

if-then-else constructs, I would extend the parser’s grammar by adding rules to the nonterminal that represents statements, and also handle the classic “dangling else” problem. For example, in a Bison-like notation, I could add:

statement : IF '(' expression ')' statement { /* if without else */ } | IF '(' expression ')' statement ELSE statement { /* if with else */ } ;

Some languages use a separate

if_stmt nonterminal and distinguish between “matched” and “unmatched”

ifs, like:

statement : matched_stmt | unmatched_stmt; matched_stmt : IF '(' expression ')' matched_stmt ELSE matched_stmt | other_stmt; unmatched_stmt : IF '(' expression ')' statement | IF '(' expression ')' matched_stmt ELSE unmatched_stmt;

This second approach makes sure each

ELSE is associated with the closest preceding unmatched

IF, resolving the dangling-else ambiguity while still allowing nested

if and

if-else constructs.